

Classification CIFAR-100

Vanilla CNN, ResNet adapté et Optimisation du processus d'apprentissage

Rapport de Projet Vision

Février 2026

1 Abstract

Ce projet explore la classification multi-classes sur des images de faible résolution **dataset CIFAR-100**. L'étude suit une progression incrémentale, partant d'une architecture **Vanilla CNN** optimisée vers une implémentation personnalisée de **ResNet**. Au-delà des architectures, ce projet se concentre également sur l'optimisation du processus d'apprentissage. **Une étude incrémentale des stratégies d'entraînement** (inspirée des améliorations de la littérature) intégrant des techniques modernes telles que le Mixup, le Cosine Annealing ou encore Sharpness-Aware Minimization (SAM). Les résultats démontrent qu'une architecture résiduelle adaptée au jeu de données, couplée à une bonne régularisation, permet de pallier les difficultés liées à la faible résolution des images et au grand nombre de classes du dataset, et d'ainsi pouvoir proposer de bien meilleurs résultats qu'une architecture basique CNN.

2 Introduction

La classification d'images sur le jeu de données **CIFAR-100** représente un défi en Computer Vision en raison du ratio élevé entre le nombre de catégories (100) et le volume d'images par classe (500 pour l'entraînement), le tout dans une résolution restreinte de 60.000 images en couleurs, de 32x32 pixels.

L'objectif de ce rapport est double :

1. **Évaluer l'apport des connexions résiduelles** par rapport à une approche convolutionnelle classique (Vanilla).
2. Quantifier le **gain de performance** apporté par les **stratégies d'optimisation de l'apprentissage** et de régularisation.

Nous détaillerons d'abord notre environnement expérimental avant de présenter l'évolution de nos modèles, de la baseline vers un système complexe optimisé.

3 Méthodologie et environnement expérimental

3.1 Jeu de données et prétraitement

Le dataset CIFAR-100 a été utilisé dans sa configuration standard. Le train set est composé de 45k images, le validation set de 5000, et le test set de 10k images. Comme il est relativement simple d'overfitter, nous avons appliqué un pipeline de Data Augmentation comprenant des transformations, afin de régulariser :

- **Géométriques :**

- Horizontal Flips : $p = 0.5$
- Vertical Flips : $p = 0.5$ (qu'on retirera avant l'étude incrémentale des stratégies d'entraînement, car il apporte du label noise en rendant certaines classes irréalistes)
- RandomCrop : 32×32 , $padding = 4$
- **Apparence :**
 - ColorJitter : 0.25 sur luminosité, contraste, saturation
- **Occlusion :**
 - RandomErasing : $p = 0.5$, scale 0.02-0.2

afin d'améliorer la robustesse des modèles.

3.2 Configuration matérielle

Tous les entraînements ont été effectués sur une carte graphique **NVIDIA RTX 3050 (8GB)**, et un processeur **AMD Ryzen 5 7600X 6-Core Processor** de 12 coeurs logiques (threads).

3.3 Protocole d'entraînement

L'optimisation repose sur la minimisation de la Cross-Entropy Loss. La gestion du taux d'apprentissage a initialement été confiée à un scheduler *ReduceLROnPlateau*, ajustant le learning rate en fonction de la stagnation de la perte de validation (*patience* = 2, *factor* = 0.5). Pour éviter le sur-apprentissage et des trainings longs et inefficaces, un EarlyStopping a été mise en place, interrompant l'entraînement après une période définie sans amélioration de la précision de validation (*patience* = 5). Il est également possible de couper l'entraînement à n'importe quel moment grâce à la mise en place d'un *ModelCheckpoint* qui sauvegarde les poids du meilleur modèle, en fonction de la loss de validation.

4 Étude des architectures baselines (Vanilla CNN)

Avant de passer à des architectures résiduelles complexes, nous avons établi un point de référence : **VanillaCNN**, un modèle baseline pour mesurer les performances d'un CNN minimaliste. Son architecture est simple :

- **Extracteur de caractéristiques :** Un bloc de trois couches de convolution successives. La première couche projette l'image d'entrée vers un espace de 32 canaux (*out_channels* = 32). Les réductions n dimensions se font à l'aide d'un bloc [Conv2d, BN, ReLU] avec un stride=2.
- **Transition :** Une opération de Flatten transforme les cartes de caractéristiques multidimensionnelles en un vecteur unidimensionnel.
- **Classifieur :** Une unique couche linéaire FC projette ce vecteur (post-Flatten) directement vers les 100 sorties (*num_classes*).

Ce modèle, bien que rapide à entraîner, souffre d'une capacité de représentation très limitée, traitant l'information de manière quasi-linéaire en fin de réseau.

Table 1: Résultats de l’entraînement du Vanilla CNN ($\text{lr_initial}=10^{-3}$, $\text{weight_decay}=10^{-4}$) obtenus en 74 epochs, interrompu par un EarlyStopping (patience=5), avec un scheduler ReduceLROnPlateau (patience=2, factor=0.5, threshold=0.001), batch de taille 128.

Architecture	Params	T_{epoch} (sec)	Top-1 Acc (%)	Top-5 Acc (%)
VanillaCNN	397,908	3.42	57.4	84.7

4.1 Premier résultat et limites

Les tests ont montré que cette architecture Vanilla plafonne rapidement :

Ces observations justifient naturellement la transition vers une architecture de type **ResNet**, capable de gérer une profondeur bien plus importante tout en facilitant la propagation du gradient, stabilisant ainsi l’entraînement. Cela permettra de complexifier le réseau, notamment avec une plus grande profondeur.

5 CIFAR-adapted ResNet

L’objectif est de surmonter le plafonnement des performances des modèles "Vanilla" en exploitant la profondeur sans subir le problème du "Vanishing gradient". En agissant comme des raccourcis pour le gradient, les connexions résiduelles préviennent ce phénomène et lissent la surface de perte (loss landscape moins "sharp"), facilitant ainsi la convergence vers un optimum global plus performant.

Le réseau peut apprendre la fonction identité si un bloc résiduel n’est pas utile (le gradient peut "court-circuiter"), ce qui revient à donner l’entrée en sortie, sans la modifier.

5.1 Implémentation du ResNet : Adaptation aux contraintes de CIFAR-100

L’architecture ResNet standard (conçue initialement pour ImageNet et des images de 224×224) est inadaptée à la faible résolution de CIFAR-100 (32×32). Appliquer le modèle de base entraînerait une réduction trop agressive de la dimension spatiale dès les premières couches.

Nous avons donc procédé à des ajustements structurels critiques :

- **Modification du bloc d’entrée (StartingBlock) :** Le noyau de 7×7 avec un stride de 2 a été remplacé par une convolution de 3×3 avec un stride de 1.
- **Suppression du MaxPool initial :** Dans le modèle original, le MaxPool réduit immédiatement la résolution. Ici, sa suppression est très importante : conserver une résolution de 32×32 au début du réseau permet de préserver l’information spatiale. Une réduction précoce à 8×8 (soit une perte de 75% de la surface de l’image avant même le premier bloc) aurait rendu l’apprentissage très compliqué.
- **Réduction de la profondeur globale :** Le quatrième bloc de couches a été supprimé pour limiter la complexité et s’adapter à la petite taille des images.

5.2 Comparaison de Profondeur : ResNet-28 vs ResNet-34

Nous avons implémenté un ResNet-28 personnalisé, structuré autour de trois séquences de blocs résiduels [3, 4, 6]. Ce modèle a été mis en concurrence avec une version adaptée (le "StartingBlock") du ResNet-34 de Torchvision, qui présente les 4 couches initiales.

Table 2: Comparaison des versions de ResNet, avec inplace=64 (Test)

Architecture	Layers - Params (en M)	T_{epoch} (sec)	Top-1 Acc (%)	Top-5 Acc (%)
ResNet (Perso. - v28)	[3, 4, 6] - 12.8M	34	70.2	92.2
ResNet (Torch - v34)	[3, 4, 6, 3] - 21.3M	44	66.9	91.0

L'analyse montre que le ResNet-28 offre un excellent compromis. Bien qu'il dispose de 40% de paramètres en moins que le ResNet-34, l'accuracy Top-1 est meilleure de +3.3%. Cela suggère qu'une profondeur excessive sur CIFAR-100 n'est pas forcément nécessaire si l'architecture est bien dimensionnée.

5.3 Résultats Intermédiaires : le saut de performance

La comparaison avec le modèle baseline (Section 4) souligne l'efficacité des connexions résiduelles, et d'un modèle plus complexe, en l'occurrence ici en largeur.

Table 3: Comparaison de ResNet28 personnalisé avec inplace=64, contre la baseline (Test set)

Architecture	T_{epoch} (sec)	Top-1 Acc (%)	Top-5 Acc (%)	Confiance Moy
Vanilla CNN (baseline)	3.42	57.4	84.7	64.7
ResNet-28 (Perso)	34	70.2	92.2	81.6

Le passage du VanillaCNN au ResNet-28 permet un gain net de 12.8% en Top-1 Acc.. La confiance moyenne et la pertinence du modèle (Top-5 à près de 90%) valident l'intérêt de cette architecture pour la suite de notre étude. On notera tout de même un temps d'entraînement qui a été multiplié quasiment par x10 pour chaque epoch.

6 Étude incrémentale des stratégies d'entraînement

L'architecture ResNet-28 fournit une base solide, l'objectif devient l'amélioration de la généralisation du modèle. Sur CIFAR-100, les performances ne dépendent pas uniquement de l'architecture : la stratégie d'entraînement, la régularisation et la gestion du taux d'apprentissage jouent un rôle déterminant. Cette section présente les différentes méthodes intégrées progressivement au pipeline d'entraînement ainsi que leur impact sur les performances.

Dans cette étude, le ResNet-14 personnalisé (layers=[2, 2, 2]) a été retenu comme modèle de référence afin de réduire le coût expérimental et accélérer les itérations (balance performance/temps).

6.1 Optimisation du taux d'apprentissage : Warmup et Cosine Annealing

Les premiers entraînements reposaient sur un scheduler *ReduceLROnPlateau*, qui réduit dynamiquement le learning rate lorsque la loss de validation stagne. Bien que simple à mettre en place, cette approche introduit des transitions abruptes du taux d'apprentissage et peut rester bloquée dans des minimas locaux "pointus".

Nous avons remplacé cette stratégie par un scheduler **Cosine Annealing** précédé d'un **Linear Warmup** sur les 10 premières epochs. Le learning rate augmente progressivement de 10^{-4} à 10^{-3} avant de décroître suivant une trajectoire cosinoïdale jusqu'à 10^{-6} en fin d'entraînement (à la dernière epoch).

Cela permet une convergence plus stable, et améliore la généralisation du modèle. Quant au warmup, il limite les instabilités initiales liées aux gradients de grande amplitude, particulièrement présentes avec les architectures résiduelles profondes et les batches importants.

6.2 Régularisation par les labels : Label Smoothing

Dans une classification classique, les labels sont encodés sous forme *one-hot*, avec une probabilité égale à 1 pour la classe cible et 0 pour les autres. Cette cible binaire, combinée à la fonction de perte **Cross-Entropy** (CE), pousse le modèle à une confiance excessive. En effet, pour minimiser $L = -\log(p_{target})$, l'optimiseur force les logits vers l'infini pour atteindre $p_{target} \approx 1$.

Le *Label Smoothing* modifie la distribution cible y en introduisant un paramètre de lissage $\alpha \in [0, 1]$. La nouvelle distribution cible y^{LS} est un mélange entre la distribution originale et une distribution uniforme sur toutes les classes K :

$$y_i^{LS} = (1 - \alpha)y_i + \frac{\alpha}{K} \quad (1)$$

Cette régularisation empêche le réseau de sur-concentrer sa probabilité sur une seule classe. Il n'est plus puni s'il n'atteint pas une certitude presque absolue ce qui limite l'apprentissage par-coeur du bruit d'entraînement. Cette régularisation favorise des représentations intra-classe plus compactes et améliore généralement la robustesse face aux ambiguïtés visuelles. Sur CIFAR-100, où certaines catégories partagent des caractéristiques proches, cette approche améliore la capacité de généralisation en réduisant la confiance excessive du modèle.

Dans nos expériences, un coefficient de `label_smoothing=0.1` a fourni les résultats les plus stables.

6.3 Augmentation de données avancée : Mixup

Le **Mixup** génère des exemples d'entraînement synthétiques par combinaisons linéaires de paires d'images et de leurs labels. Pour deux échantillons (x_i, y_i) et (x_j, y_j) , on construit :

$$\tilde{x} = \lambda x_i + (1 - \lambda)x_j \quad (2)$$

$$\tilde{y} = \lambda y_i + (1 - \lambda)y_j \quad (3)$$

avec $\lambda \sim \text{Beta}(\alpha, \alpha)$.

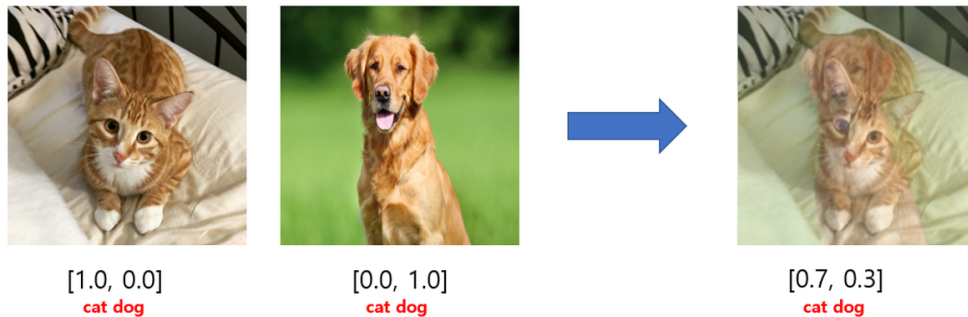


Figure 1: Illustration du mécanisme Mixup : création d'un exemple hybride à partir de deux images et de leurs labels. Source : Medium - [Paper] Mixup: Beyond Empirical Risk Minimization (Image Classification).

Contrairement au Label Smoothing, qui agit uniquement sur les cibles, le Mixup modifie directement la distribution des données d'entraînement. Le modèle apprend ainsi des transitions continues entre les classes plutôt que des frontières de décision trop rigides.

L'intégration du Mixup nécessite également une adaptation de la fonction de perte :

$$L(\tilde{x}) = \lambda CE(pred, y_i) + (1 - \lambda) CE(pred, y_j) \quad (4)$$

Cette approche transforme la cible en une distribution continue (*Soft Labels*), ce qui réduit la sur-confiance du réseau et améliore significativement sa robustesse aux exemples ambigus. Dans nos expériences, le Mixup apporte un gain plus important que le Label Smoothing seul, au prix d'une légère augmentation du temps d'entraînement.

En revanche, l'ajout du Mixup nécessite un ajustement du *Label Smoothing*. En effet, les labels produits par le Mixup sont déjà des *soft labels*. Conserver un coefficient de lissage trop élevé introduit alors une double régularisation des cibles, pouvant ralentir l'apprentissage et dégrader la séparation entre classes. Le coefficient de `label_smoothing` a donc été réduit de 0.1 à 0.05 en conséquence.

6.4 Optimisation de la géométrie de la loss : Sharpness-Aware Minimization (SAM)

L'impact de **Sharpness-Aware Minimization (SAM)** a été étudiée, une méthode d'optimisation visant à améliorer la généralisation en recherchant des minimas de loss plus plats.

Contrairement à une optimisation classique, SAM ne minimise pas uniquement la perte au point courant des poids w , mais cherche une solution robuste aux petites perturbations locales des poids. À chaque itération, l'algorithme effectue une première passe pour identifier la direction maximisant localement la perte, puis une seconde mise à jour est réalisée afin de minimiser cette perte perturbée.

En pratique, cette stratégie régularise fortement l'entraînement et réduit la sensibilité du modèle aux minima trop "pointus", souvent associés à une moins bonne généralisation.

L'utilisation de SAM a nécessité plusieurs ajustements expérimentaux :

- réduction du learning rate initial à 3×10^{-4} ;
- réduction du weight decay ;
- conservation du Label Smoothing afin de stabiliser davantage les prédictions.

SAM et Mixup n'ont pas été combinés dans cette étude. Compatibles en théorie, ces deux méthodes introduisent tout deux une forte régularisation. Leur association nécessite un réajustement fin des hyperparamètres afin d'éviter une sous-optimisation du modèle. Afin de conserver une étude incrémentale contrôlée et comparable, SAM a donc été évalué indépendamment du pipeline Mixup.

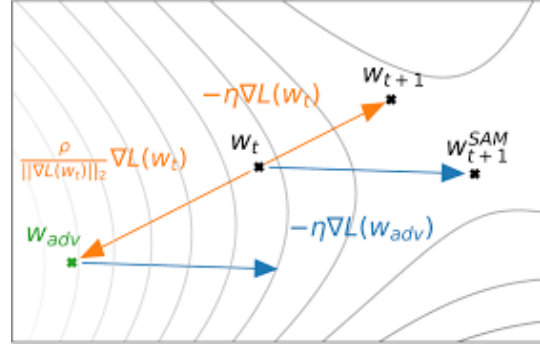
Attention, SAM est très coûteux computationnellement : chaque itération nécessite une double passe forward/backward.

6.5 Synthèse de l'étude

Le tableau ci-dessous résume l'évolution des performances, à mesure que nous intégrons ces stratégies au pipeline d'entraînement. Sauf mention contraire, les expériences utilisent un batch size de 256,



(a) A gauche un minimum local "sharp", à droite un minimum local dont le paysage de la loss est plus lissé ("smooth") représentant le type de paysage obtenu avec SAM.



(b) On calcule le gradient dont les poids associés sont w_{t+1} , on applique une perturbation aux poids w_t avec le gradient précédemment calculé pour obtenir w_{adv} les poids perturbés, puis on obtient une direction en optimisant depuis w_{adv} , qu'on applique aux poids initiaux w_t .

Figure 2: (a) : Visualisation de la loss landscape sans et avec SAM. (b) : Schéma descriptif des étapes de SAM.

l'optimiseur AdamW, un entraînement sur 150 epochs, et une régularisation renforcée par un `weight_decay` désormais à 10^{-3} (contre 10^{-4} précédemment).

Table 4: Impact incrémental des stratégies d'entraînement sur le ResNet-14 personnalisé, avec un learning rate initial égal au weight decay, de 10^{-3}

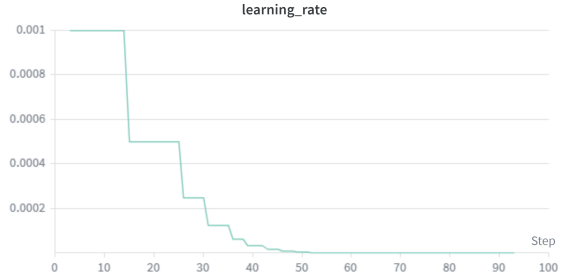
Configuration	Top-1 Acc (%)	Gain	T_{epoch} (sec)	Informations
ResNet-14 (baseline)	68.9	/	30.33	ReduceLROnPlateau
+ Cosine Annealing & Label Smoothing	73.2	+4.3	30.17	Linear Warmup + LR cosinoïdal, $\alpha_{LS} = 0.1$
+ SE Blocks	73.3	+4.4	33.29	ratio=4
+ Mixup	74.6	+5.7	33.20	$\alpha_{Mixup} = 0.2, \alpha_{LS} = 0.05$

Table 5: Impact incrémental des stratégies d'entraînement sur le ResNet-14 personnalisé avec SAM (sans Mixup) et avec un learning rate initial égal au weight decay de 3×10^{-4} .

Configuration	Top-1 Acc (%)	Gain (Acc.)	T_{epoch} (sec)	Informations
ResNet-14 (Perso)	68.9	/	30.33	ReduceLROnPlateau
+ Cosine Annealing & Label Smoothing + SE Blocks + SAM	74.5	+5.6	63.51	$\alpha_{LS} = 0.1$

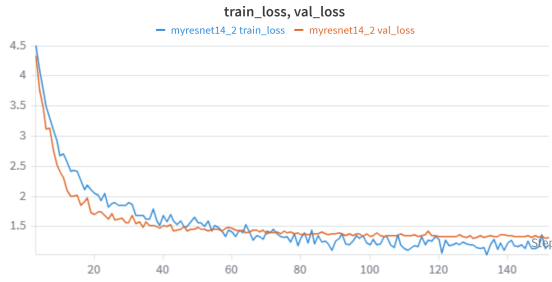


(a)

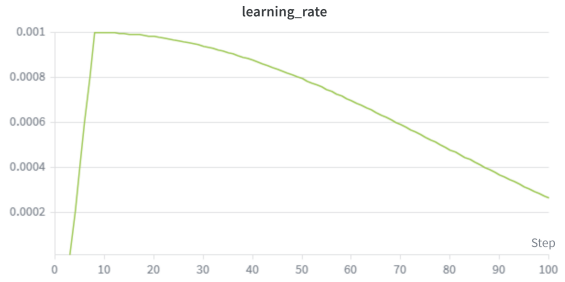


(b)

Figure 3: (a) : Train & Val losses de ResNet-14 (b) : Évolution du learning rate avec le scheduler ReduceLROnPlateau, de ResNet-14.



(a)



(b)

Figure 4: (a) : Train & Val losses de la meilleure configuration de ResNet-14 (avec Mixup) (b) : Évolution du learning rate avec le scheduler combinant Linear Warmup et Cosine Annealing, de la meilleure configuration de ResNet-14 (avec Mixup).

7 Conclusion

L'amélioration progressive du pipeline d'entraînement permet un gain total de 5.7 points de précision Top-1 par rapport au ResNet-14 de base. Les résultats montrent que les stratégies d'optimisation et de régularisation ont un impact comparable à celui des modifications architecturales.

À partir d'un ResNet-14 personnalisé atteignant 68.9% de Top-1, l'ajout du Cosine Annealing couplé au Label Smoothing apporte déjà un gain substantiel de +4.3 points avec un coût computationnel négligeable. Cela souligne l'importance de la dynamique du taux d'apprentissage et de la régularisation sur la stabilité de convergence. L'ajout des blocs SE ne se traduisent ensuite que par une amélioration marginale (+0.1), ce qui témoigne de leur contribution fortement dépendante du contexte d'optimisation et probablement limitée dans une architecture de cette taille, surtout lorsqu'elle est déjà bien régularisée. On peut aussi se questionner quant à son utilité au regard du jeu de données, avec des images faibles résolutions, focalisées sur un élément, où l'on peut penser que chaque channel compte pour l'analyse des représentations.

L'introduction du Mixup constitue l'un des facteurs le plus déterminant dans ce premier protocole, avec un gain final de +5.7 points pour atteindre 74.6%. Cette amélioration de 1.3% par rapport à la configuration précédente indique que la régularisation par combinaisons linéaires des exemples renforce significativement la robustesse du modèle, notamment face aux frontières de classes ambiguës.

Évalué séparément, l'intégration de SAM en remplacement du Mixup (et avec un learning rate initial plus faible à 3×10^{-4}) conduit à une performance semblable en terme de Top-1 Acc. (74.5%, +5.6), mais avec un coût computationnel nettement supérieur (63.51s contre 33s par epoch). Cela traduit un compromis clair : SAM permet une optimisation robuste, mais au prix d'un coût computationnel doublé. Cependant, la différence de learning rate entre les deux expériences introduit un biais expérimental rendant difficile une comparaison stricte. Cette méthode doit être considérée selon les contraintes du projet, le jeu de données, et les besoins en précision finale.

En résumé, un gain significatif de +11.5% a été apporté par le changement d'architecture, en passant d'un CNN minimaliste (Vanilla CNN) à un ResNet dont l'architecture a été adaptée au jeu de données (ResNet-14). Les stratégies d'optimisation et de régularisation ont également eu un impact important, signant un gain de +5.7%, permettant à une configuration avec un petit modèle ResNet d'atteindre les 74.5% de Top-1 Acc. sur la classification de 10k images composées de 100 classes.